

Enunciado – Entrega 1 de Algoritmos e Lógica de Programação

Professor Dr. Marcelo Amorim.

1. Contexto e objetivo

Nesta entrega, cada grupo desenvolve em C# o programa **Relatório da Partida**, que lê os dados de uma partida do quiz *ARCOR – Desafio das Marcas* e apresenta um resumo estatístico. A entrega vale **3,0 pontos** e o prazo é Oct 9, 2026, até 23:59:56.

O programa é uma peça do próprio PI: ele corresponde ao resultado final da partida (RF12 do briefing) e será ampliado na Entrega 2, com modularização, vetores e matrizes.

O objetivo é aplicar, num problema real do projeto, as três estruturas desta etapa:

- **Sequencial:** leitura dos dados e cálculos.
- **Decisão:** validação das entradas, classificação e escolhas do relatório.
- **Repetição:** gráfico de asteriscos do desempenho por nível.

Também faz parte do objetivo testar o programa, comparar o resultado obtido com o esperado e registrar os ajustes feitos quando os dois divergirem.

2. Visão geral do programa

Cada execução do programa trata **uma única partida**: lê 10 dados em sequência, valida cada um, faz os cálculos e imprime o relatório. O fluxo é sempre este:

1. Ler uma entrada com `Console.ReadLine()`.
2. Validar essa entrada logo em seguida, antes de ler a próxima.
3. Se a entrada for inválida, mostrar a mensagem de erro e encerrar o programa com `Environment.Exit(1)`, sem gerar relatório.
4. Repetir os passos 1 a 3 para as 10 entradas, na ordem da seção 3.
5. Com todas as entradas válidas, fazer os cálculos da seção 5.
6. Imprimir o relatório no modelo da seção 6.

O passo 4 não é um laço: são 10 blocos de leitura escritos um depois do outro. A estrutura de repetição desta entrega aparece no gráfico de asteriscos (seções 5 e 6).

3. Entradas

O programa lê exatamente 10 entradas, uma por linha, **nesta ordem**. Por privacidade (requisito do briefing da Arcor), o jogador é identificado apenas por nickname e faixa etária.

Nº	Entrada	Tipo	Exemplo
1	Nickname	texto	Tortuguita123
2	Faixa etária (código do menu abaixo)	inteiro	3
3	Questões fáceis apresentadas	inteiro	4
4	Acertos nas fáceis	inteiro	4
5	Questões médias apresentadas	inteiro	4
6	Acertos nas médias	inteiro	3
7	Questões difíceis apresentadas	inteiro	2
8	Acertos nas difíceis	inteiro	1
9	Tempo total da partida, em segundos	inteiro	180
10	Dicas usadas na partida	inteiro	1

Menu de faixa etária (entrada 2):

Código	Faixa
1	Até 12 anos
2	13 a 17 anos
3	18 a 24 anos
4	25 a 39 anos
5	40 anos ou mais
6	Prefiro não informar

Antes de cada leitura, o programa pode mostrar uma mensagem pedindo o dado (por exemplo, Acertos nas fáceis:). O arquivo de entrada do exemplo acima fica assim, uma linha por dado:

```
Tortuguital23
3
4
4
4
3
2
1
180
1
```

4. Validação e tratamento de erros

Na **primeira** entrada inválida, o programa mostra uma mensagem de erro e encerra, sem gerar o relatório. Ele nunca pode "quebrar" com uma exceção.

4.1 Regras de validação

Entrada	Regra	Exemplo inválido
1. Nickname	Não pode ser vazio	linha em branco
2. Faixa etária	Número inteiro de 1 a 6	7, abc
3, 5 e 7. Questões por nível	Número inteiro, zero ou maior	-2, dez
4, 6 e 8. Acertos por nível	Número inteiro, zero ou maior, e não maior que as questões daquele nível	5 acertos em 4 fáceis
7. Total geral	A soma das questões dos três níveis deve ser pelo menos 1	0 + 0 + 0
9. Tempo	Número inteiro maior que zero	0, -30
10. Dicas	Número inteiro, zero ou maior, e não maior que o total de questões	-1; 15 dicas em 10 questões

A regra do total geral é verificada logo após a entrada 7 e protege os cálculos: sem ela, o percentual geral dividiria por zero. Um nível com zero questões **é válido** (o jogador pode não ter recebido questões difíceis, por exemplo).

4.2 Mensagem de erro padronizada

Toda mensagem de erro segue este formato:

ERRO na entrada <número> (<nome da entrada>): <motivo>.

Exemplos:

ERRO na entrada 2 (faixa etária): valor fora do intervalo de 1 a 6.
ERRO na entrada 4 (acertos fáceis): valor maior que o total de questões fáceis.
ERRO na entrada 7 (questões difíceis): a partida precisa ter pelo menos 1 questão.

4.3 Lendo a linha: Console.ReadLine() e o string?

O Console.ReadLine() sempre lê a linha como **texto** e não gera erro, mesmo que o conteúdo esteja errado ("abc", "-5" ou uma linha vazia são textos válidos). Quando não há mais linhas para ler (por exemplo, o arquivo de entrada acabou antes da hora), ele devolve null.

Por isso, o Visual Studio mostra um aviso amarelo se você declarar string linha. Use string? linha: o ? indica ao compilador que a variável pode ficar nula. Não se preocupe com o null na conversão: o TryParse trata esse caso.

4.4 Convertendo para número: int.TryParse

O int.TryParse tenta converter o texto em número inteiro e devolve true (conseguiu) ou false (não conseguiu). O número convertido sai pela variável indicada com out. Veja a forma em passos separados:

```
string? linha = Console.ReadLine();
int acertosFaceis;
bool converteu = int.TryParse(linha, out acertosFaceis);

if (!converteu)
{
    Console.WriteLine("ERRO na entrada 4 (acertos fáceis): valor '" + linha +
        "' mal formado.");
    Environment.Exit(1);
}
```

O que acontece em cada linha:

1. Console.ReadLine() lê a linha como texto e guarda em linha.
2. int acertosFaceis; declara a variável que vai receber o número.

3. `int.TryParse(linha, out acertosFaceis)` tenta converter. Se conseguir, grava o número em `acertosFaceis` e devolve `true`. Se não conseguir, grava `0` e devolve `false`.
4. O `if` testa o resultado e, se a conversão falhou, mostra a mensagem e encerra.

A mesma coisa pode ser escrita de forma resumida. Aqui o `out int` **declara e preenche** a variável dentro dos parênteses, e ela continua disponível depois do `if`:

```
if (!int.TryParse(Console.ReadLine(), out int acertosFaceis))
{
    Console.WriteLine("ERRO na entrada 4 (acertos fáceis): valor mal formado.");
    Environment.Exit(1);
}
```

O que o `TryParse` já trata sozinho: linha vazia, texto no lugar de número e fim do arquivo (`null`). Os três casos devolvem `false`.

O que o `TryParse` NÃO trata: valor negativo, valor fora da faixa (faixa etária 7) ou valor incoerente com outra entrada (acertos maiores que questões). Para isso, use um segundo `if`, depois da conversão:

```
if (acertosFaceis < 0 || acertosFaceis > questoesFaceis)
{
    Console.WriteLine("ERRO na entrada 4 (acertos fáceis): valor fora do intervalo de 0 a " + questoesFaceis + ".");
    Environment.Exit(1);
}
```

Não use `int.Parse` nem `Convert.ToInt32`: com um dado errado, eles lançam uma exceção e o programa termina de forma abrupta, com uma mensagem técnica em inglês. É exatamente o que esta entrega pede para evitar.

4.5 Encerrando o programa: `Environment.Exit`

`Environment.Exit(código)` encerra o programa imediatamente, de qualquer ponto do código. Por convenção, o código `0` indica sucesso e qualquer outro valor indica erro. Nesta entrega, use `Environment.Exit(1)` em todos os erros de validação.

Para conferir o código de saída logo depois de executar, digite `echo $?` (Linux/Mac) ou `echo %ERRORLEVEL%` (Prompt de Comando do Windows).

4.6 Opcional: mensagem específica para dado ausente

Se o arquivo acabar antes da hora, a mensagem padrão sai como `valor '' mal formado`, porque o `null` vira texto vazio. Para uma mensagem mais precisa, verifique o `null` antes da conversão:

```

if (linha == null)
{
    Console.WriteLine("ERRO na entrada 4 (acertos fáceis): dado ausente (fim do arquivo).");
    Environment.Exit(1);
}

```

5. Cálculos e regras fixas

Todos os valores e limites desta seção são **fixos**: não podem ser alterados pelo grupo. Assim, o mesmo arquivo de entrada gera o mesmo relatório em qualquer grupo.

5.1 Totais e percentuais

- **Total de questões** = fáceis + médias + difíceis.
- **Total de acertos** = acertos fáceis + acertos médios + acertos difíceis.
- **Total de erros** = total de questões – total de acertos.
- **Percentual de um nível** = acertos do nível $\times 100,0 \div$ questões do nível.
- **Percentual geral** = total de acertos $\times 100,0 \div$ total de questões.

Atenção à divisão inteira. Em C#, a divisão entre dois int descarta a parte decimal: $1 / 2$ resulta em 0, e $1 / 2 * 100$ também dá 0. Escreva `acertos * 100.0 / questoes`: o `100.0` força a conta a ser feita com casas decimais. Guarde o resultado em uma variável `double`.

Nível não jogado. Se as questões de um nível forem zero, **não calcule** o percentual daquele nível (seria divisão por zero). No relatório, o nível aparece como "não jogado".

5.2 Pontuação

Item	Pontos
Acerto em questão fácil	+10
Acerto em questão média	+20
Acerto em questão difícil	+30
Cada dica usada	–5

- **Pontuação** = acertos fáceis $\times 10$ + acertos médios $\times 20$ + acertos difíceis $\times 30$ – dicas $\times 5$.
- Se a pontuação ficar negativa, ela passa a valer **0**.
- **Pontuação máxima** = questões fáceis $\times 10$ + questões médias $\times 20$ + questões difíceis $\times 30$ (como se o jogador acertasse tudo sem usar dicas).
- **Aproveitamento da pontuação** = pontuação $\times 100,0 \div$ pontuação máxima.

5.3 Tempo

- **Minutos** = tempo ÷ 60 (divisão inteira) e **segundos restantes** = tempo % 60.
Exemplo: 185 s = 3 min 5 s.
- **Tempo médio por questão** = tempo × 1,0 ÷ total de questões, em segundos (com casas decimais).

5.4 Faixa etária

Converta o código lido (1 a 6) no texto da faixa, conforme o menu da seção 3, usando `switch` ou `if/else if`.

5.5 Classificação (pelo percentual geral)

Percentual geral	Classificação
90,0% ou mais	Mestre das Marcas
70,0% a menos de 90,0%	Conhecedor de Marcas
50,0% a menos de 70,0%	Aprendiz
Menos de 50,0%	Iniciante

5.6 Ritmo (pelo tempo médio por questão)

Tempo médio por questão	Ritmo
Até 10,0 s	Rápido
Mais de 10,0 s até 20,0 s	Normal
Mais de 20,0 s	Pausado

5.7 Melhor nível

O melhor nível é o de **maior percentual** entre os níveis jogados.

- Compare os percentuais "na mão", com `if`. Não use `Math.Max`.
- Ignore os níveis não jogados (zero questões).
- **Empate:** vence o nível mais difícil. Exemplo: fácil com 100,0% e difícil com 100,0% → melhor nível é "Difícil".

5.8 Gráfico de desempenho (estrutura de repetição)

Para cada nível jogado, imprima um asterisco (*) por acerto, usando um laço for.

Exemplo: 3 acertos → ***. Um nível jogado com 0 acertos não mostra asteriscos. Este é o trecho obrigatório de repetição da entrega.

6. Modelo de saída do relatório

O relatório deve seguir **exatamente** este modelo: mesmas linhas, mesma ordem, mesmos textos e mesmos valores. Diferenças apenas na quantidade de espaços de alinhamento não são penalizadas.

Exemplo 1 — entrada da seção 3 (fácil 4/4, médio 3/4, difícil 1/2, 180 s, 1 dica):

```
===== ARCOR – DESAFIO DAS MARCAS: RESUMO DA PARTIDA =====
```

```
Jogador: Tortuguital23
```

```
Faixa etária: 18 a 24 anos
```

```
Desempenho por nível:
```

```
Fácil    (4/4)  100,0%  ****
```

```
Médio   (3/4)   75,0%  ***
```

```
Difícil (1/2)   50,0%  *
```

```
Total: 8 acertos e 2 erros em 10 questões (80,0%)
```

```
Pontuação: 125 de 180 pontos possíveis (69,4%)
```

```
Dicas usadas: 1 (penalidade de 5 pontos)
```

```
Tempo total: 3 min 0 s | Média: 18,0 s por questão
```

```
Ritmo: Normal
```

```
Melhor nível: Fácil
```

```
Classificação: Conhecedor de Marcas
```

```
=====
```

Exemplo 2 — nível não jogado e empate no melhor nível (fácil 5/5, médio 5/5, difícil 0/0, 50 s, 0 dicas, faixa 6):

```
===== ARCOR – DESAFIO DAS MARCAS: RESUMO DA PARTIDA =====
```

```
Jogador: TriunfoFan
```

```
Faixa etária: Prefiro não informar
```

```
Desempenho por nível:
```

```
Fácil    (5/5)  100,0%  *****
```

```
Médio   (5/5)  100,0%  *****
```

```
Difícil (0/0)  não jogado
```

```
Total: 10 acertos e 0 erros em 10 questões (100,0%)
```

```
Pontuação: 150 de 150 pontos possíveis (100,0%)
```

```
Dicas usadas: 0 (penalidade de 0 pontos)
```

```
Tempo total: 0 min 50 s | Média: 5,0 s por questão
```

```
Ritmo: Rápido
```

```
Melhor nível: Médio
```

```
Classificação: Mestre das Marcas
```

```
=====
```

No exemplo 2, fácil e médio empatam com 100,0%, e o médio vence por ser o mais difícil entre os jogadores.

Formatação com uma casa decimal. Use `{valor:F1}` dentro de uma string interpolada, por exemplo `${percentual:F1}%`. Isso é apenas apresentação e **não** conta como atalho matemático. O separador decimal (vírgula ou ponto) depende da configuração regional do computador; os dois são aceitos.

7. Como executar e testar

O programa é escrito uma única vez, com `Console.ReadLine()`, e pode ser executado de duas formas sem nenhuma mudança no código: digitando os dados no teclado ou lendo de um arquivo `.txt`.

7.1 Pelo teclado

No terminal, dentro da pasta do projeto:

```
dotnet run
```

Digite cada entrada e tecle Enter. Útil para os primeiros testes.

7.2 Por arquivo (redirecionamento de entrada)

O símbolo `<` faz o programa ler as linhas do arquivo como se tivessem sido digitadas:

```
dotnet run < testes/valido01.txt
```

Esse comando funciona no **Prompt de Comando (cmd)** do Windows e no terminal do Linux/Mac. No **PowerShell**, o `<` não funciona; use:

```
Get-Content testes\valido01.txt | dotnet run
```

No PowerShell, o código de saída é consultado com `$LASTEXITCODE`.

7.3 Arquivos de teste obrigatórios

Cada caso de teste é um arquivo `.txt` com as 10 linhas de entrada, guardado na pasta testes da entrega. O grupo deve criar **no mínimo**:

- **3 arquivos válidos** (`valido01.txt` a `valido03.txt`): juntos, devem gerar pelo menos duas classificações diferentes e incluir um nível não jogado. Os arquivos do grupo devem ser diferentes dos arquivos de exemplo da seção 7.4.
- **5 arquivos inválidos** (`erro01.txt` a `erro05.txt`), cada um violando uma regra **diferente** da seção 4.1.

Para cada arquivo, o grupo registra no documento de entrega a saída esperada (calculada à mão, antes de executar) e a saída obtida. Quando as duas forem diferentes, o grupo descreve a causa e a correção feita.

7.4 Arquivos de exemplo fornecidos

O professor disponibiliza o pacote `exemplos_entrega1.zip` com 4 entradas de exemplo e a saída esperada de cada uma. Use esses arquivos para conferir o programa antes de criar os testes do grupo.

Arquivo	Situação testada	Código de saída
<code>exemplo01.txt</code>	Partida comum (exemplo 1 da seção 6)	0
<code>exemplo02.txt</code>	Nível não jogado e empate no melhor nível (exemplo 2 da seção 6)	0
<code>exemplo03.txt</code>	Pontuação negativa que vira zero, classificação Iniciante, ritmo Pausado e níveis com 0 acertos	0
<code>exemplo04.txt</code>	Acertos médios maiores que as questões médias (erro na entrada 6)	1

Cada `exemploNN.txt` tem um `exemploNN_saida_esperada.txt` correspondente. Se o programa mostra mensagens pedindo os dados, elas aparecem antes do relatório, o que é esperado. No `exemplo04`, a parte `ERR0` na entrada 6 (acertos médios): deve ser igual; o motivo depois dos dois-pontos pode ter outra redação.

Os exemplos servem apenas para conferência. Eles não contam como testes do grupo e não podem ser copiados para a pasta testes.

8. Restrições

Esta entrega avalia o uso das estruturas sequencial, de decisão e de repetição. Por isso, os cálculos devem ser feitos "na mão", com operadores e estruturas básicas.

Permitido	Proibido
Variáveis <code>int</code> , <code>double</code> , <code>string</code> , <code>bool</code>	Vetores, listas e matrizes (ficam para a

Permitido	Proibido
	Entrega 2)
Operadores + - * / % e de comparação	Math.Max, Math.Min, Math.Round e outros métodos de Math
if, else if, else, switch	LINQ (Sum(), Average(), Max() e similares)
for, while, do-while	int.Parse e Convert.ToInt32 (encerram o programa com exceção)
Console.ReadLine(), Console.WriteLine()	Leitura de arquivo pelo código (File.ReadAllLines e similares); use o redirecionamento da seção 7
int.TryParse, Environment.Exit	Bibliotecas ou pacotes externos
Formatação {valor:F1}	

O código fica todo no programa principal (Program.cs), sem funções criadas pelo grupo. A modularização é o tema da Entrega 2.

9. O que entregar e onde

A entrega é feita no repositório do grupo no GitHub, até Oct 9, 2026 às 23:59:56, e tem três partes: o documento de entrega em PDF, o código Program.cs e os arquivos de teste. Atrasos perdem 25% da nota por dia.

9.1 Estrutura da pasta

Tudo fica dentro da pasta **Algoritmos e Lógica de Programação**, na subpasta **Entrega 1**:

```
Algoritmos e Lógica de Programação/
├── Entrega 1/
│   ├── Entrega1_<NomeDoGrupo>.pdf
│   ├── Program.cs
│   └── testes/
│       ├── valido01.txt
│       ├── valido02.txt
│       ├── valido03.txt
│       ├── erro01.txt
│       ├── erro02.txt
│       ├── erro03.txt
│       ├── erro04.txt
│       └── erro05.txt
```

9.2 Documento de entrega (PDF)

O grupo preenche o **documento-modelo (template)** disponibilizado pelo professor e o exporta em PDF. O template já traz as seções a preencher:

- Identificação: nome do projeto, professor orientador e integrantes do grupo.
- Trechos do código que mostram cada estrutura (sequencial, decisão e repetição), com a explicação de entrada e saída de cada trecho.
- Tabela de casos de teste: arquivo, saída esperada, saída obtida e resultado.
- Divergências encontradas e ajustes feitos.
- Observações reflexivas do grupo.

No PDF entram apenas trechos do código, nunca o código inteiro. O código completo fica somente no arquivo `Program.cs`.

9.3 Código e testes

- O `Program.cs` deve compilar e executar com `dotnet run`, sem erros.
- Os arquivos da pasta `testes` devem ser exatamente os usados na tabela de casos de teste do PDF. O professor executará o programa com esses arquivos para conferir os resultados.

10. Checklist antes de entregar

- ☐ O programa lê as 10 entradas na ordem da seção 3.
- ☐ Todas as conversões usam `int.TryParse`, e a variável da linha é `string?`.
- ☐ Todas as regras da seção 4.1 estão validadas, com a mensagem no formato padronizado.
- ☐ Todo erro encerra o programa com `Environment.Exit(1)`, sem exceção.
- ☐ Os cálculos usam os valores fixos da seção 5 e evitam a divisão inteira.
- ☐ Níveis não jogados aparecem como "não jogado".
- ☐ O gráfico de asteriscos usa `for`.
- ☐ Nenhum item proibido da seção 8 aparece no código.
- ☐ Com os 4 arquivos de exemplo da seção 7.4, as saídas são iguais às saídas esperadas.
- ☐ Há pelo menos 3 arquivos válidos e 5 inválidos na pasta `testes`.
- ☐ O PDF foi feito a partir do template, com identificação do grupo e sem o código inteiro.
- ☐ PDF, `Program.cs` e pasta `testes` estão em `Algoritmos e Lógica de Programação/Entrega 1/` no GitHub.